JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Format

java.lang.Object
    java.text.Format

All Implemented Interfaces:

`Serializable`, `Cloneable`

Direct Known Subclasses:

`DateFormat`, `ListFormat`, `MessageFormat`, `NumberFormat`

```
public abstract class Format
extends Object
implements Serializable, Cloneable
```

`Format` is an abstract base class for formatting locale-sensitive information such as dates, messages, numbers, and lists.

`Format` defines the programming interface for formatting locale-sensitive objects into `Strings` (the `format` method) and for parsing `Strings` back into objects (the `parseObject` method).

Generally, a format's `parseObject` method must be able to parse any string formatted by its `format` method. However, there may be exceptional cases where this is not possible. For example, a `format` method might create two adjacent integer numbers with no separator in between, and in this case the `parseObject` could not tell which digits belong to which number.

Subclassing

The Java Platform provides specialized subclasses of `Format`-- `DateFormat`, `MessageFormat`, `NumberFormat`, and `ListFormat`--for formatting dates, messages, numbers, and lists respectively.

Concrete subclasses must implement three methods:

1. `format(Object obj, StringBuffer toAppendTo, FieldPosition pos)`
2. `formatToCharacterIterator(Object obj)`
3. `parseObject(String source, ParsePosition pos)`

These general methods allow polymorphic parsing and formatting of objects and are used, for example, by `MessageFormat`. Subclasses often also provide additional `format` methods for specific input types as well as `parse` methods for specific result types. Any `parse` method that does not take a `ParsePosition` argument should throw `ParseException` when no text in the required format is at the beginning of the input text.

Most subclasses will also implement the following factory methods:

1. `getInstance` for getting a useful format object appropriate for the current locale
2. `getInstance(Locale)` for getting a useful format object appropriate for the specified locale

In addition, some subclasses may also implement other `getXxxxInstance` methods for more specialized control. For example, the `NumberFormat` class provides `getPercentInstance` and `getCurrencyInstance` methods for getting specialized number formatters.

Subclasses of `Format` that allow programmers to create objects for locales (with `getInstance(Locale)` for example) must also implement the following class method:

```
public static Locale[] getAvailableLocales()
```

Subclasses may also consider implementing leniency when parsing. The definition of leniency should be delegated to the subclass.

And finally subclasses may define a set of constants to identify the various fields in the formatted output. These constants are used to create a FieldPosition object which identifies what information is contained in the field and its position in the formatted result. These constants should be named *`item`*`_FIELD` where *`item`* identifies the field. For examples of these constants, see `ERA_FIELD` and its friends in `DateFormat`.

Synchronization

Formats are generally not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

Since:

1.1

See Also:

`ParsePosition`, `FieldPosition`, `NumberFormat`, `DateFormat`, `MessageFormat`, `ListFormat`, Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description

static class	Format.Field	Defines constants that are used as attribute keys in the AttributedCharacterIterator returned from Format.formatToCharacterIterator and as field identifiers in FieldPosition.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	Format()	Sole constructor.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
Object	clone()	Creates and returns a copy of this object.
final String	format(Object obj)	Formats an object to produce a string.
abstract StringBuffer	format(Object obj, StringBuffer toAppendTo, FieldPosition pos)	Formats an object and appends the resulting text to a given string buffer.
AttributedCharacterI	formatToCharacterIterato (Object obj)	Formats an Object producing an AttributedCharacterIterator.
Object	parseObject (String source)	Parses text from the beginning of the given string to produce an object.
abstract Object		

	parseObject(String source, ParsePosition pos)	Parses text from the given string to produce an object.

Methods declared in class Object

equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Format

```
protected Format()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Details

format

```
public final String format(Object obj)
```

Formats an object to produce a string.

Implementation Requirements:

This method returns a string that would be equal to the string returned by

```
format(obj, new StringBuffer(), new
FieldPosition(0)).toString();
```

Parameters:

obj - The object to format

Returns:

Formatted string.

Throws:

IllegalArgumentException - if the Format cannot format the given object

format

```
public abstract StringBuffer format(Object obj,
                                    StringBuffer toAppendTo,
                                    FieldPosition pos)
```

Formats an object and appends the resulting text to a given string buffer. If the `pos` argument identifies a field used by the format, then its indices are set to the beginning and end of the first such field encountered.

Parameters:

`obj` - The object to format

`toAppendTo` - where the text is to be appended

`pos` - A `FieldPosition` identifying a field in the formatted text

Returns:

the string buffer passed in as `toAppendTo`, with formatted text appended

Throws:

`NullPointerException` - if `toAppendTo` or `pos` is null

`IllegalArgumentException` - if the Format cannot format the given object

formatToCharacterIterator

```
public AttributedCharacterIterator formatToCharacterIterator
(Object obj)
```

Formats an Object producing an `AttributedCharacterIterator`. You can use the returned `AttributedCharacterIterator` to build the resulting String, as well as to determine information about the resulting String.

Each attribute key of the AttributedCharacterIterator will be of type `Field`. It is up to each `Format` implementation to define what the legal values are for each attribute in the `AttributedCharacterIterator`, but typically the attribute key is also used as the attribute value.

API Note:

Subclasses that support fields should override this and create an `AttributedCharacterIterator` with meaningful attributes.

Implementation Requirements:

The default implementation creates an `AttributedCharacterIterator` with no attributes.

Parameters:

`obj` - The object to format

Returns:

AttributedCharacterIterator describing the formatted value.

Throws:

`NullPointerException` - if obj is null.

IllegalArgumentException - when the Format cannot format the given object.

Since:

1.4

parseObject

```
public abstract Object parseObject(String source,
                                   ParsePosition pos)
```

Parses text from the given string to produce an object.

This method attempts to parse text starting at the index given by `pos`. If parsing succeeds, then the index of `pos` is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed object is returned. The updated `pos` can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of `pos` is not changed, the error index of `pos` is set to the index of the character where the error occurred, and `null` is returned.

Parameters:

`source` - the `String` to parse

`pos` - A `ParsePosition` object with index and error index information as described above.

Returns:

An `Object` parsed from the string. In case of error, returns `null`.

Throws:

`NullPointerException` - if `source` or `pos` is `null`.

parseObject

```
public Object parseObject(String source)
                  throws ParseException
```

Parses text from the beginning of the given string to produce an object. This method may not use the entire text of the given string.

Parameters:

`source` - A `String`, to be parsed from the beginning.

Returns:

An `Object` parsed from the string.

Throws:

`ParseException` - if parsing fails

`NullPointerException` - if `source` is `null`.

clone

```
public Object clone()
```

Creates and returns a copy of this object.

Overrides:

`clone` in class `Object`

Returns:

a clone of this instance.

See Also:

`Cloneable`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.